

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC - KỸ THUẬT MÁY TÍNH



Mật mã và An ninh mạng (CO3069)

HIỆN THỰC HỆ MÃ RSA
SỬ DỤNG SỐ NGUYÊN TỔ LỚN

GVHD: Nguyễn Cao Đạt

STT	Họ tên SV	MSSV
1	Hoàng Đình Anh Trung	2213683
2	Nguyễn Phương Hoài Vũ	2213999
3	Nguyễn Minh Minh Nguyên	2212309

TP. Hồ Chí Minh, Tháng 05/2026

Mục lục

1	Giới thiệu	3
1.1	Bối cảnh bài tập lớn	3
1.2	Yêu cầu của bài tập lớn	3
1.3	Mục tiêu thực hiện	4
1.4	Phạm vi thực hiện	4
1.5	Phương pháp thực hiện	5
1.6	Cấu trúc báo cáo	5
2	Cơ sở lý thuyết	6
2.1	Mã hóa khóa công khai	6
2.2	Số nguyên tố và bài toán phân tích thừa số	6
2.3	Ước số chung lớn nhất và nguyên tố cùng nhau	6
2.4	Hàm phi Euler	7
2.5	Nghịch đảo modulo và thuật toán Euclid mở rộng	7
2.6	Kiểm tra số nguyên tố và thuật toán Miller-Rabin	7
2.7	Lũy thừa modulo và thuật toán Square and Multiply	8
2.8	Quy trình tạo khóa RSA	8
2.9	Mã hóa và giải mã RSA	9
2.10	Cơ sở an toàn của RSA	9
3	Phân tích và thiết kế	11
3.1	Phân tích yêu cầu	11
3.2	Thiết kế tổng quan hệ thống	11
3.3	Luồng xử lý tạo khóa	12
3.4	Luồng xử lý mã hóa	12
3.5	Luồng xử lý giải mã	13
3.6	Thiết kế dữ liệu đầu vào và đầu ra	13
3.7	Định hướng hiện thực	13
4	Hiện thực và đánh giá	15
4.1	Môi trường hiện thực	15
4.2	Các chức năng đã hiện thực dự kiến	15
4.3	Hiện thực các thuật toán chính	16
4.4	Quy trình kiểm thử dự kiến	16
4.5	Kết quả đánh giá dự kiến	16
4.6	Nhận xét sơ bộ	17



5	Kết luận	18
5.1	Kết quả đạt được dự kiến	18
5.2	Ưu điểm	18
5.3	Hạn chế	18
5.4	Hướng phát triển	19
5.5	Kết luận chung	19
6	Mã nguồn dự án	20

1 Giới thiệu

1.1 Bối cảnh bài tập lớn

Trong lĩnh vực an toàn thông tin, mật mã học đóng vai trò rất quan trọng trong việc bảo vệ dữ liệu trong quá trình lưu trữ và truyền tải. Một trong những hệ mật mã khóa công khai phổ biến nhất là RSA. Hệ mã RSA được sử dụng rộng rãi trong nhiều ứng dụng thực tế như chữ ký số, trao đổi khóa, bảo mật giao tiếp trên Internet và xác thực dữ liệu.

Khác với các hệ mật mã khóa đối xứng, RSA sử dụng hai khóa khác nhau: khóa công khai dùng để mã hóa và khóa bí mật dùng để giải mã. Độ an toàn của RSA dựa trên độ khó của bài toán phân tích một số nguyên rất lớn thành tích của hai số nguyên tố lớn. Vì vậy, để hiểu rõ cơ chế hoạt động của RSA, việc tự hiện thực các bước tính toán cơ bản là rất cần thiết.

Trong bài tập lớn này, nhóm tiến hành hiện thực hệ mã RSA bằng ngôn ngữ lập trình đã chọn. Nhóm không sử dụng các thư viện hoặc hàm RSA có sẵn. Thay vào đó, nhóm tự xây dựng các hàm cần thiết như sinh số nguyên tố lớn, tính ước số chung lớn nhất, tạo khóa, mã hóa và giải mã thông điệp.

1.2 Yêu cầu của bài tập lớn

Bài tập lớn yêu cầu nhóm hiện thực hệ mã RSA trên một trong các ngôn ngữ lập trình Java, C, C++ hoặc Python. Trong quá trình hiện thực, nhóm được phép sử dụng kiểu dữ liệu số nguyên lớn để lưu trữ và tính toán với các số có kích thước lớn. Tuy nhiên, nhóm không được sử dụng trực tiếp các hàm đã hiện thực sẵn các thuật toán chính như tính ước số chung lớn nhất, lũy thừa modulo, kiểm tra số nguyên tố hoặc tạo khóa RSA.

Theo yêu cầu của đề bài, các số nguyên tố lớn được sử dụng trong hệ RSA phải có kích thước ít nhất 500 bits. Chương trình cần hiện thực các chức năng chính sau:

- Tìm số nguyên tố lớn khi cho trước số lượng bits của số nguyên tố cần tìm.
- Tính ước số chung lớn nhất của hai số nguyên lớn.
- Tính khóa giải mã d khi biết khóa mã hóa e và hai số nguyên tố lớn.
- Tạo bộ khóa ngẫu nhiên khi cho trước hai số nguyên tố lớn.
- Mã hóa thông điệp khi biết thông điệp, khóa mã hóa e và modulo n .
- Giải mã thông điệp khi biết bản mã, khóa giải mã d và modulo n .

Các yêu cầu trên giúp nhóm hiểu rõ hơn về các thành phần toán học cốt lõi của hệ mã RSA, bao gồm số nguyên tố lớn, hàm Euler, modulo, thuật toán Euclid, thuật toán Euclid mở rộng và phép lũy thừa modulo.

1.3 Mục tiêu thực hiện

Mục tiêu chính của bài tập lớn là xây dựng một chương trình RSA hoàn chỉnh ở mức cơ bản, có khả năng tạo khóa, mã hóa và giải mã thông điệp. Thông qua quá trình hiện thực, nhóm hướng đến các mục tiêu cụ thể sau:

- Hiểu được nguyên lý hoạt động của hệ mật mã RSA.
- Nắm được vai trò của hai số nguyên tố lớn p và q trong quá trình tạo khóa.
- Hiện thực được các thuật toán toán học cần thiết mà không phụ thuộc vào thư viện RSA có sẵn.
- Kiểm tra được tính đúng đắn của quá trình mã hóa và giải mã.
- Đánh giá được ưu điểm, hạn chế và hướng phát triển của chương trình.

Bên cạnh việc xây dựng chương trình, bài tập lớn cũng giúp nhóm rèn luyện khả năng phân tích yêu cầu, thiết kế thuật toán, tổ chức mã nguồn và trình bày báo cáo kỹ thuật.

1.4 Phạm vi thực hiện

Trong phạm vi bài tập lớn này, nhóm tập trung vào việc hiện thực các chức năng cơ bản của hệ mã RSA. Chương trình được xây dựng nhằm minh họa quá trình hoạt động của RSA từ bước sinh số nguyên tố, tạo khóa cho đến mã hóa và giải mã.

Phạm vi thực hiện bao gồm:

- Làm việc với số nguyên lớn có kích thước tối thiểu 500 bits.
- Tự hiện thực các thuật toán cơ bản như kiểm tra số nguyên tố, tính ước số chung lớn nhất, tìm nghịch đảo modulo và lũy thừa modulo.
- Tạo khóa công khai và khóa bí mật dựa trên hai số nguyên tố lớn.
- Mã hóa và giải mã thông điệp dạng số nguyên.
- Thử nghiệm chương trình với một số trường hợp đầu vào khác nhau để kiểm tra kết quả.

Tuy nhiên, chương trình chủ yếu phục vụ mục đích học tập và minh họa thuật toán. Vì vậy, chương trình chưa hướng đến việc thay thế các thư viện RSA chuẩn trong các hệ thống thực tế. Một số vấn đề bảo mật nâng cao như padding, chống tấn công kênh bên, quản lý khóa an toàn và tối ưu hóa hiệu năng ở mức hệ thống chưa được triển khai đầy đủ trong phạm vi bài tập này.

1.5 Phương pháp thực hiện

Để hoàn thành bài tập lớn, nhóm thực hiện theo các bước chính sau:

1. Tìm hiểu cơ sở lý thuyết về hệ mã RSA và các thuật toán toán học liên quan.
2. Phân tích yêu cầu của đề bài và xác định các chức năng cần hiện thực.
3. Thiết kế cấu trúc chương trình và luồng xử lý cho từng chức năng.
4. Hiện thực các thuật toán cơ bản như Euclid, Euclid mở rộng, kiểm tra số nguyên tố và lũy thừa modulo.
5. Xây dựng các hàm tạo khóa, mã hóa và giải mã RSA.
6. Kiểm thử chương trình với các đầu vào khác nhau.
7. Đánh giá kết quả đạt được, ưu điểm, hạn chế và đề xuất hướng phát triển.

Cách tiếp cận này giúp nhóm vừa đảm bảo đáp ứng yêu cầu kỹ thuật của đề bài, vừa hiểu rõ vai trò của từng thành phần trong hệ mã RSA.

1.6 Cấu trúc báo cáo

Báo cáo kỹ thuật được tổ chức thành năm chương chính như sau:

- **Chương 1 – Giới thiệu:** Trình bày bối cảnh, yêu cầu, mục tiêu, phạm vi và phương pháp thực hiện bài tập lớn.
- **Chương 2 – Cơ sở lý thuyết:** Trình bày các kiến thức nền tảng liên quan đến hệ mã RSA, bao gồm mật mã khóa công khai, số nguyên tố lớn, modulo, hàm Euler, thuật toán Euclid và các bước hoạt động của RSA.
- **Chương 3 – Phân tích và thiết kế:** Phân tích yêu cầu bài toán, thiết kế các chức năng chính, mô tả luồng xử lý và cấu trúc chương trình.
- **Chương 4 – Hiện thực và đánh giá:** Trình bày cách hiện thực chương trình, mô tả các hàm chính, kết quả chạy thử và đánh giá chương trình.
- **Chương 5 – Kết luận:** Tổng kết những nội dung đã thực hiện, nêu ưu điểm, hạn chế và hướng phát triển trong tương lai.

Ngoài các chương chính, báo cáo còn bao gồm phần tài liệu tham khảo và phụ lục. Phần phụ lục trình bày thêm thông tin về môi trường phát triển, thư viện hỗ trợ và các tài liệu đã sử dụng trong quá trình thực hiện bài tập lớn.

2 Cơ sở lý thuyết

2.1 Mã hóa khóa công khai

Mã hóa khóa công khai là hệ mã sử dụng hai khóa khác nhau: khóa công khai và khóa riêng. Khóa công khai có thể được chia sẻ cho người khác để mã hóa thông điệp, trong khi khóa riêng được giữ bí mật để giải mã. Đây là điểm khác biệt chính so với mã hóa khóa đối xứng, nơi cả bên gửi và bên nhận dùng cùng một khóa.

Trong bài tập lớn này, nhóm tập trung hiện thực hệ mã RSA. RSA là một trong những hệ mã khóa công khai phổ biến, dựa trên các khái niệm lý thuyết số như số nguyên tố lớn, phép modulo, hàm phi Euler và bài toán phân tích thừa số nguyên tố.

2.2 Số nguyên tố và bài toán phân tích thừa số

Số nguyên tố là số tự nhiên lớn hơn 1 và chỉ chia hết cho 1 và chính nó. Trong RSA, hai số nguyên tố lớn p và q được chọn để tạo ra modulo:

$$n = p \times q$$

Việc nhân hai số nguyên tố lớn để tạo ra n là dễ thực hiện. Tuy nhiên, nếu chỉ biết n , việc phân tích ngược lại thành p và q là rất khó khi các số này đủ lớn. Đây là nền tảng an toàn chính của RSA.

Theo yêu cầu của đề bài, các số nguyên tố được sử dụng phải có kích thước ít nhất 500 bits. Vì vậy, chương trình cần có chức năng sinh số nguyên lớn ngẫu nhiên và kiểm tra xem số đó có phải là số nguyên tố hay không.

2.3 Ước số chung lớn nhất và nguyên tố cùng nhau

Ước số chung lớn nhất của hai số a và b , ký hiệu là $\gcd(a, b)$, là số nguyên dương lớn nhất chia hết cho cả a và b . Hai số được gọi là nguyên tố cùng nhau nếu:

$$\gcd(a, b) = 1$$

Trong RSA, điều kiện này được dùng khi chọn số mũ mã hóa e . Cụ thể, e phải thỏa mãn:

$$1 < e < \varphi(n)$$

và:

$$\gcd(e, \varphi(n)) = 1$$

Điều kiện trên đảm bảo rằng tồn tại khóa giải mã d , là nghịch đảo modulo của e theo modulo $\varphi(n)$.

Để tính gcd, chương trình sử dụng thuật toán Euclid dựa trên công thức:

$$\gcd(a, b) = \gcd(b, a \bmod b)$$

Thuật toán được lặp lại cho đến khi phần dư bằng 0.

2.4 Hàm phi Euler

Hàm phi Euler, ký hiệu là $\varphi(n)$, cho biết số lượng các số nguyên dương nhỏ hơn n và nguyên tố cùng nhau với n .

Trong RSA, vì:

$$n = p \times q$$

với p và q là hai số nguyên tố khác nhau, nên:

$$\varphi(n) = (p - 1)(q - 1)$$

Giá trị $\varphi(n)$ được dùng trong quá trình chọn khóa mã hóa e và tính khóa giải mã d . Nếu kẻ tấn công biết được $\varphi(n)$, họ có thể tìm được khóa riêng, vì vậy p , q và $\varphi(n)$ cần được giữ bí mật.

2.5 Nghịch đảo modulo và thuật toán Euclid mở rộng

Trong RSA, khóa giải mã d được tính sao cho:

$$e \times d \equiv 1 \pmod{\varphi(n)}$$

Điều này có nghĩa là d là nghịch đảo modulo của e theo modulo $\varphi(n)$.

Để tìm d , chương trình sử dụng thuật toán Euclid mở rộng. Thuật toán này không chỉ tính $\gcd(a, b)$ mà còn tìm được hai số nguyên x và y sao cho:

$$ax + by = \gcd(a, b)$$

Khi $\gcd(e, \varphi(n)) = 1$, thuật toán Euclid mở rộng cho phép tìm được d thỏa mãn điều kiện cần thiết để giải mã RSA.

2.6 Kiểm tra số nguyên tố và thuật toán Miller-Rabin

Do RSA cần sử dụng các số nguyên tố rất lớn, phương pháp chia thử truyền thống không phù hợp vì thời gian chạy quá lâu. Vì vậy, chương trình sử dụng thuật toán Miller-Rabin để kiểm tra tính nguyên tố.

Miller-Rabin là thuật toán kiểm tra nguyên tố xác suất dựa trên các tính chất của số nguyên tố và định lý Fermat. Với một số lẻ $n > 2$, ta viết:

$$n - 1 = 2^k q$$

trong đó q là số lẻ. Sau đó, thuật toán chọn một số ngẫu nhiên a với:

$$1 < a < n - 1$$

và kiểm tra các điều kiện modulo để xác định n là hợp số hay có khả năng là số nguyên tố.

Nếu thuật toán trả về “hợp số”, thì số đó chắc chắn là hợp số. Nếu trả về “có thể là số nguyên tố”, chương trình sẽ lặp lại nhiều lần với các giá trị a khác nhau để tăng độ tin cậy. Cách này phù hợp với yêu cầu sinh số nguyên tố lớn trong RSA.

2.7 Lũy thừa modulo và thuật toán Square and Multiply

Trong RSA, mã hóa và giải mã đều cần tính các biểu thức dạng:

$$a^b \bmod n$$

Nếu tính trực tiếp a^b rồi mới lấy modulo thì kết quả trung gian sẽ rất lớn và không hiệu quả. Vì vậy, chương trình cần dùng thuật toán lũy thừa modulo nhanh, thường gọi là Square and Multiply.

Ý tưởng của thuật toán là biểu diễn số mũ b dưới dạng nhị phân. Sau đó, chương trình lần lượt bình phương kết quả và nhân thêm với cơ sở khi bit hiện tại bằng 1. Sau mỗi bước, kết quả đều được lấy modulo n để tránh số trung gian quá lớn.

Thuật toán này giúp quá trình mã hóa và giải mã RSA chạy nhanh hơn, đặc biệt khi làm việc với các số có kích thước hàng trăm bits.

2.8 Quy trình tạo khóa RSA

Quá trình tạo khóa RSA gồm các bước chính sau:

1. Chọn ngẫu nhiên hai số nguyên tố lớn p và q , với $p \neq q$.
2. Tính:

$$n = p \times q$$

3. Tính:

$$\varphi(n) = (p - 1)(q - 1)$$

4. Chọn số mũ mã hóa e sao cho:

$$1 < e < \varphi(n), \quad \gcd(e, \varphi(n)) = 1$$

5. Tính số mũ giải mã d sao cho:

$$e \times d \equiv 1 \pmod{\varphi(n)}$$

Sau khi hoàn tất, ta có:

$$PU = \{e, n\}$$

là khóa công khai, và:

$$PR = \{d, n\}$$

là khóa riêng.

2.9 Mã hóa và giải mã RSA

Để mã hóa một thông điệp M , trước hết thông điệp cần được biểu diễn dưới dạng số nguyên và phải thỏa mãn:

$$0 \leq M < n$$

Sau đó, dùng khóa công khai $PU = \{e, n\}$ để tính bản mã:

$$C = M^e \bmod n$$

Để giải mã, dùng khóa riêng $PR = \{d, n\}$ để khôi phục thông điệp:

$$M = C^d \bmod n$$

Nếu quá trình tạo khóa đúng, kết quả giải mã sẽ trùng với thông điệp ban đầu.

2.10 Cơ sở an toàn của RSA

Độ an toàn của RSA dựa trên bài toán phân tích thừa số nguyên tố. Cụ thể, từ hai số nguyên tố lớn p và q , việc tính $n = p \times q$ là dễ. Tuy nhiên, từ n để tìm lại p và q là rất khó nếu kích thước khóa đủ lớn.

Nếu kẻ tấn công phân tích được n thành p và q , họ có thể tính được:

$$\varphi(n) = (p-1)(q-1)$$

và từ đó tìm được khóa riêng d . Vì vậy, trong quá trình hiện thực, việc sinh số nguyên tố đủ lớn và giữ bí mật p , q , $\varphi(n)$ là rất quan trọng.



Trong phạm vi bài tập lớn, chương trình tập trung vào việc hiện thực đúng các thành phần cơ bản của RSA. Các vấn đề bảo mật nâng cao như chống tấn công thời gian, chống tấn công được chọn bản mã hoặc padding an toàn chưa được triển khai đầy đủ.

3 Phân tích và thiết kế

3.1 Phân tích yêu cầu

Bài tập lớn yêu cầu nhóm hiện thực hệ mã RSA bằng một ngôn ngữ lập trình như Java, C, C++ hoặc Python. Trong quá trình hiện thực, nhóm không sử dụng các thư viện RSA có sẵn mà phải tự xây dựng các hàm tính toán chính cần thiết cho RSA.

Các chức năng chính cần hiện thực bao gồm:

- Sinh số nguyên tố lớn theo số lượng bits cho trước.
- Tính ước số chung lớn nhất của hai số nguyên lớn.
- Tính khóa giải mã d khi biết khóa mã hóa e và hai số nguyên tố lớn p, q .
- Tạo bộ khóa RSA ngẫu nhiên từ hai số nguyên tố lớn.
- Mã hóa thông điệp bằng khóa công khai (e, n) .
- Giải mã bản mã bằng khóa riêng (d, n) .

Vì RSA làm việc với các số nguyên rất lớn, chương trình cần hỗ trợ kiểu dữ liệu số nguyên lớn. Tuy nhiên, các thuật toán chính như kiểm tra số nguyên tố, tính GCD, tìm nghịch đảo modulo và lũy thừa modulo sẽ được nhóm tự hiện thực thay vì dùng trực tiếp các hàm có sẵn.

3.2 Thiết kế tổng quan hệ thống

Hệ thống được thiết kế theo hướng chia nhỏ thành các nhóm chức năng chính. Mỗi nhóm chức năng đảm nhiệm một phần cụ thể trong quá trình hoạt động của RSA.

Các thành phần chính của chương trình gồm:

- **Khối xử lý số học:** hiện thực các phép toán cơ bản như GCD, Euclid mở rộng, nghịch đảo modulo và lũy thừa modulo.
- **Khối kiểm tra và sinh số nguyên tố:** sinh số ngẫu nhiên có số bits yêu cầu và kiểm tra tính nguyên tố bằng thuật toán Miller-Rabin.
- **Khối tạo khóa RSA:** sử dụng hai số nguyên tố p, q để tính $n, \varphi(n)$, chọn e và tính d .
- **Khối mã hóa:** nhận thông điệp và khóa công khai để tạo bản mã.
- **Khối giải mã:** nhận bản mã và khóa riêng để khôi phục thông điệp ban đầu.

Cách chia này giúp chương trình dễ kiểm thử, dễ sửa lỗi và dễ mở rộng trong các phiên bản tiếp theo.

3.3 Luồng xử lý tạo khóa

Quá trình tạo khóa RSA được thiết kế theo các bước sau:

1. Nhập hoặc sinh ngẫu nhiên hai số nguyên tố lớn p và q .
2. Kiểm tra điều kiện $p \neq q$.
3. Tính modulo:

$$n = p \times q$$

4. Tính hàm phi Euler:

$$\varphi(n) = (p - 1)(q - 1)$$

5. Chọn số mũ mã hóa e sao cho:

$$1 < e < \varphi(n), \quad \gcd(e, \varphi(n)) = 1$$

6. Tính khóa giải mã d bằng nghịch đảo modulo:

$$d \equiv e^{-1} \pmod{\varphi(n)}$$

7. Xuất khóa công khai (e, n) và khóa riêng (d, n) .

Trong thiết kế ban đầu, nhóm có thể chọn giá trị $e = 65537$ vì đây là giá trị thường dùng trong RSA. Nếu e không nguyên tố cùng nhau với $\varphi(n)$, chương trình sẽ chọn lại e hoặc sinh lại cặp số nguyên tố p, q .

3.4 Luồng xử lý mã hóa

Quá trình mã hóa nhận đầu vào là thông điệp M và khóa công khai (e, n) . Thông điệp cần được chuyển sang dạng số nguyên và thỏa mãn điều kiện:

$$0 \leq M < n$$

Sau đó, chương trình tính bản mã:

$$C = M^e \pmod{n}$$

Phép tính này được thực hiện bằng thuật toán lũy thừa modulo nhanh để tránh việc tạo ra số trung gian quá lớn.

3.5 Luồng xử lý giải mã

Quá trình giải mã nhận đầu vào là bản mã C và khóa riêng (d, n) . Chương trình tính lại thông điệp ban đầu bằng công thức:

$$M = C^d \bmod n$$

Sau khi giải mã, kết quả dạng số nguyên có thể được chuyển lại về dạng thông điệp ban đầu nếu chương trình có hỗ trợ chuyển đổi giữa chuỗi ký tự và số nguyên.

3.6 Thiết kế dữ liệu đầu vào và đầu ra

Đầu vào của chương trình gồm:

- Số bits dùng để sinh số nguyên tố lớn.
- Hai số nguyên tố p, q nếu người dùng muốn nhập thủ công.
- Thông điệp cần mã hóa.
- Khóa công khai (e, n) hoặc khóa riêng (d, n) tùy chức năng.

Đầu ra của chương trình gồm:

- Hai số nguyên tố lớn p, q .
- Khóa công khai (e, n) .
- Khóa riêng (d, n) .
- Bản mã sau khi mã hóa.
- Thông điệp sau khi giải mã.

3.7 Định hướng hiện thực

Trong giai đoạn hiện thực, nhóm dự kiến xây dựng chương trình theo từng hàm riêng biệt. Mỗi hàm sẽ xử lý một chức năng nhỏ để thuận tiện cho việc kiểm thử.

Các hàm chính dự kiến gồm:

- `gcd(a, b)`: tính ước số chung lớn nhất.
- `extended_gcd(a, b)`: hiện thực thuật toán Euclid mở rộng.
- `mod_inverse(e, phi)`: tính nghịch đảo modulo.
- `mod_pow(a, b, n)`: tính lũy thừa modulo nhanh.



- `is_prime(n)`: kiểm tra tính nguyên tố.
- `generate_prime(bits)`: sinh số nguyên tố lớn.
- `generate_keys(p, q)`: tạo khóa RSA.
- `encrypt(M, e, n)`: mã hóa thông điệp.
- `decrypt(C, d, n)`: giải mã bản mã.

Thiết kế này giúp nhóm có thể kiểm tra từng thuật toán riêng lẻ trước khi ghép lại thành chương trình RSA hoàn chỉnh.

4 Hiện thực và đánh giá

4.1 Môi trường hiện thực

Chương trình dự kiến được hiện thực bằng ngôn ngữ lập trình Python. Lý do chọn Python là vì ngôn ngữ này hỗ trợ kiểu số nguyên lớn một cách tự nhiên, giúp thuận tiện khi làm việc với các số có kích thước hàng trăm bits.

Môi trường phát triển dự kiến:

- Ngôn ngữ lập trình: Python.
- Công cụ phát triển: Visual Studio Code hoặc Google Colab.
- Hệ điều hành: Windows hoặc môi trường trực tuyến Google Colab.
- Thư viện hỗ trợ: chỉ sử dụng thư viện cơ bản để sinh số ngẫu nhiên và xử lý đầu vào/đầu ra.

Theo yêu cầu của đề bài, nhóm không sử dụng các thư viện RSA có sẵn. Các thuật toán chính phục vụ RSA sẽ được nhóm tự hiện thực.

4.2 Các chức năng đã hiện thực dự kiến

Trong phiên bản đầu tiên, chương trình dự kiến hiện thực các chức năng chính sau:

- Sinh số nguyên tố lớn theo số bits nhập vào.
- Kiểm tra tính nguyên tố bằng thuật toán Miller-Rabin.
- Tính ước số chung lớn nhất bằng thuật toán Euclid.
- Tính nghịch đảo modulo bằng thuật toán Euclid mở rộng.
- Tạo khóa công khai và khóa riêng RSA.
- Mã hóa thông điệp dạng số nguyên.
- Giải mã bản mã để khôi phục thông điệp ban đầu.

Các chức năng này tương ứng với các yêu cầu chính của bài tập lớn và là nền tảng để xây dựng hệ mã RSA hoàn chỉnh.

4.3 Hiện thực các thuật toán chính

Đối với hàm tính GCD, chương trình sử dụng thuật toán Euclid. Thuật toán này có ưu điểm là đơn giản, dễ hiện thực và xử lý hiệu quả với số nguyên lớn.

Đối với hàm tính khóa giải mã d , chương trình sử dụng thuật toán Euclid mở rộng để tìm nghịch đảo modulo của e theo modulo $\varphi(n)$.

Đối với phép tính lũy thừa modulo trong mã hóa và giải mã, chương trình sử dụng thuật toán Square and Multiply. Thuật toán này giúp giảm thời gian xử lý so với cách tính lũy thừa trực tiếp.

Đối với chức năng sinh số nguyên tố lớn, chương trình sinh số ngẫu nhiên có kích thước bits theo yêu cầu, sau đó kiểm tra tính nguyên tố bằng thuật toán Miller-Rabin. Nếu số sinh ra chưa phải số nguyên tố, chương trình tiếp tục sinh hoặc kiểm tra số khác cho đến khi tìm được số phù hợp.

4.4 Quy trình kiểm thử dự kiến

Sau khi hiện thực, chương trình sẽ được kiểm thử theo từng nhóm chức năng.

Trước hết, các hàm toán học cơ bản như `gcd`, `extended_gcd`, `mod_inverse` và `mod_pow` sẽ được kiểm tra với các số nhỏ để dễ đối chiếu kết quả thủ công.

Sau đó, chức năng tạo khóa RSA sẽ được kiểm tra bằng cách nhập hai số nguyên tố nhỏ, ví dụ $p = 17$ và $q = 11$, để so sánh với ví dụ lý thuyết. Khi kết quả đúng, chương trình sẽ tiếp tục được thử nghiệm với các số nguyên tố lớn hơn.

Cuối cùng, quá trình mã hóa và giải mã sẽ được kiểm tra bằng nguyên tắc:

$$\text{Decrypt}(\text{Encrypt}(M)) = M$$

Nếu thông điệp sau giải mã trùng với thông điệp ban đầu, chương trình được xem là hoạt động đúng đối với trường hợp kiểm thử đó.

4.5 Kết quả đánh giá dự kiến

Do chương trình hiện chưa hoàn thiện đầy đủ, phần kết quả đánh giá sẽ được bổ sung sau khi nhóm hoàn tất hiện thực và chạy thử.

Các tiêu chí đánh giá dự kiến gồm:

- Chương trình có sinh được số nguyên tố lớn đúng số bits yêu cầu hay không.
- Hàm GCD và nghịch đảo modulo có cho kết quả đúng hay không.
- Bộ khóa RSA được tạo ra có hợp lệ hay không.
- Thông điệp sau khi giải mã có trùng với thông điệp ban đầu hay không.
- Thời gian xử lý khi sử dụng số nguyên tố có kích thước từ 500 bits trở lên.



Bảng kết quả kiểm thử sẽ được bổ sung sau khi chương trình hoàn tất:

Trường hợp	Đầu vào	Kết quả mong đợi	Kết quả thực tế
1	Số nhỏ kiểm thử RSA	Giải mã đúng thông điệp	Bổ sung sau
2	Số nguyên tố 500 bits	Tạo khóa thành công	Bổ sung sau
3	Mã hóa thông điệp	Tạo bản mã hợp lệ	Bổ sung sau
4	Giải mã bản mã	Khôi phục thông điệp ban đầu	Bổ sung sau

Bảng 1: Bảng kiểm thử dự kiến cho chương trình RSA

4.6 Nhận xét sơ bộ

Về mặt thiết kế, chương trình có thể đáp ứng các yêu cầu chính của đề bài nếu các thuật toán được hiện thực đúng. Việc chia chương trình thành nhiều hàm nhỏ giúp quá trình kiểm thử dễ dàng hơn.

Tuy nhiên, do RSA làm việc với số nguyên rất lớn, thời gian sinh số nguyên tố và thời gian mã hóa/giải mã có thể tăng khi kích thước khóa lớn. Ngoài ra, phiên bản hiện thực trong bài tập lớn chủ yếu phục vụ mục đích học tập, chưa bao gồm đầy đủ các cơ chế bảo mật nâng cao như padding an toàn hoặc chống tấn công thời gian.

5 Kết luận

5.1 Kết quả đạt được dự kiến

Trong bài tập lớn này, nhóm tìm hiểu và hiện thực hệ mã RSA dựa trên các kiến thức về mã hóa khóa công khai và lý thuyết số. Các nội dung chính bao gồm số nguyên tố lớn, ước số chung lớn nhất, hàm phi Euler, nghịch đảo modulo, lũy thừa modulo và quy trình tạo khóa RSA.

Về mặt thiết kế, nhóm đã xác định được các chức năng cần có của chương trình, bao gồm sinh số nguyên tố lớn, tạo khóa, mã hóa và giải mã. Các thuật toán chính cũng đã được lựa chọn để phục vụ quá trình hiện thực, bao gồm thuật toán Euclid, Euclid mở rộng, Miller-Rabin và Square and Multiply.

Sau khi hoàn tất chương trình, nhóm kỳ vọng hệ thống có thể tạo được bộ khóa RSA hợp lệ, mã hóa thông điệp bằng khóa công khai và giải mã chính xác bằng khóa riêng.

5.2 Ưu điểm

Thiết kế của chương trình có một số ưu điểm như sau:

- Các chức năng được chia thành từng hàm riêng biệt, giúp dễ kiểm thử và sửa lỗi.
- Chương trình không phụ thuộc vào thư viện RSA có sẵn, phù hợp với yêu cầu của đề bài.
- Các thuật toán được lựa chọn phù hợp với việc xử lý số nguyên lớn.
- Cấu trúc chương trình có thể mở rộng thêm các chức năng khác trong tương lai.

5.3 Hạn chế

Do phạm vi bài tập lớn chủ yếu tập trung vào việc hiện thực cơ chế RSA cơ bản, chương trình vẫn còn một số hạn chế:

- Chưa triển khai đầy đủ các cơ chế padding an toàn cho RSA.
- Chưa tối ưu sâu về hiệu năng khi xử lý số nguyên có kích thước rất lớn.
- Chưa có cơ chế bảo vệ trước các dạng tấn công nâng cao như tấn công thời gian hoặc tấn công được chọn bản mã.
- Việc mã hóa thông điệp hiện chỉ tập trung vào dạng số nguyên hoặc dạng chuyển đổi đơn giản.

Các hạn chế này sẽ được xem xét và cập nhật lại sau khi nhóm hoàn tất phần hiện thực và kiểm thử thực tế.

5.4 Hướng phát triển

Trong tương lai, chương trình có thể được phát triển thêm theo các hướng sau:

- Hỗ trợ mã hóa và giải mã chuỗi văn bản hoàn chỉnh thay vì chỉ xử lý số nguyên.
- Bổ sung cơ chế chia nhỏ thông điệp khi thông điệp lớn hơn n .
- Thêm padding an toàn như OAEP để tăng khả năng chống tấn công.
- Tối ưu thời gian sinh số nguyên tố lớn và thời gian mã hóa/giải mã.
- Xây dựng giao diện dòng lệnh hoặc giao diện người dùng đơn giản để dễ sử dụng hơn.
- Bổ sung thêm bộ kiểm thử tự động để đánh giá tính đúng đắn của chương trình.

5.5 Kết luận chung

Tóm lại, bài tập lớn giúp nhóm hiểu rõ hơn về nguyên lý hoạt động của hệ mã RSA và các kiến thức toán học liên quan. Thông qua việc tự thiết kế và hiện thực các thuật toán cần thiết, nhóm có thể nắm được cách tạo khóa, mã hóa và giải mã trong hệ mã khóa công khai.

Mặc dù chương trình hiện mới hướng đến mục đích học tập và minh họa thuật toán, đây là nền tảng quan trọng để nhóm tiếp tục hoàn thiện hệ thống RSA đầy đủ hơn trong các bước tiếp theo.



6 Mã nguồn dự án

Toàn bộ mã nguồn của dự án được triển khai và chạy thử nghiệm trên nền tảng Google Colab. Chi tiết có thể được xem tại các đường dẫn dưới đây:

- Colab: [BTL Hệ mã RSA trên Python](#)



Tài liệu

- [1] Nguyễn Cao Đạt (2024), *Slide bài giảng môn Mật mã và An ninh mạng*, Trường Đại học Bách Khoa – Đại học Quốc gia TP.HCM.
- [2] William Stallings (2023), *Cryptography and Network Security*, 8th Edition, Prentice Hall.
- [3] Wolfram MathWorld. "Euclidean Algorithm". <https://mathworld.wolfram.com/EuclideanAlgorithm.html>
- [4] Wikipedia. "Miller–Rabin Primality Test". https://en.wikipedia.org/wiki/Miller%E2%80%93Rabin_primality_test